Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.9 / COMPANION WORKBOOK

Build the Linux repository on SBD-REPO01

Mirrored, signature-verified package snapshots for the Linux servers

Build the one approved source of operating-system packages before any Linux server exists: verify the vendor's signing keys, build and harden SBD-REPO01 on VLAN 70 as a minimal web server, mirror and publish dated snapshots carried across the air gap and checked by hash and signature, write the client configuration every Linux server will use, and prove promotion, rollback and the first STIG baseline with the SCC for Linux set every Linux server is scanned with.

MATERIALS EDITION 1.0 / COURSE 1.9

Updated 14 October 2026 · sbd-linux-repository

NAME _____ / DATE _____

© 2026 Secure By Design Team LLC.

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Design the repository and its trust

02 Build and harden SBD-REPO01

03 Mirror, verify, and publish a snapshot

04 Configure the Linux clients

05 Keep it current and prove it

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

© 2026 Secure By Design Team LLC.

Before you start

SBD-REPO01 is the Linux side's one road for operating-system packages, built before any Linux server exists so that SBD-SPLUNK01 (course 1.12) patches from it from its first day. It also stages the Linux assessment set, SCC for Linux and the distribution's STIG benchmark, that every Linux server is scanned with. Every Linux server in the lab runs a Red Hat Enterprise Linux–family distribution, so the commands use dnf, rpm, firewalld and update-ca-trust; choose one release for every Linux server and follow it throughout.

Prerequisites

- ☐ Complete courses 1.1–1.8: the domain with DNS and course 1.3's time design with its chrony file, the course 1.4 media procedure and DISA tooling, course 1.6's root certificate and the hash it recorded, and the tiered administration plane, with SBD-ADMIN-SRV01 as the Tier 1 seat.
- ☐ Provision a virtual machine for SBD-REPO01 on VLAN 70 (10.0.70.20) with a separate data disk for /srv (it holds /srv/repo, /srv/incoming and /srv/repo-records), and install media for the one Red Hat Enterprise Linux–family release your Linux servers will use, with the vendor's published checksum.
- ☐ Have an approved connected system that can reach the distribution's repositories (for Red Hat Enterprise Linux, a registered system of the same release and architecture) and the removable-media procedure for carrying the mirror across.

What this course builds

- Design the Linux package source: one Red Hat Enterprise Linux–family distribution, the repositories mirrored, verified upstream keys, and the register rows.
- Build SBD-REPO01 as a hardened, minimal web server administered only from the Tier 1 seat.
- Mirror, verify and publish dated snapshots, with promotion and rollback on record.
- Hand later courses a tested client configuration that trusts only SBD-REPO01 and the verified keys.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-REPO01 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 25 MIN

Design the repository and its trust

WHY THIS MATTERS Before anything is mirrored, decide what SBD-REPO01 is for: the one approved source of operating-system packages for the domain-joined Linux servers, fed by dated snapshots whose upstream signing keys you verified first. Then place it on VLAN 70 and write its rows into the register.

In this lesson you will

- Define the repository's job, its consumers, and the one-source rule for Linux packages.
- Choose what is mirrored and verify the upstream signing keys that make it trustworthy.
- Place SBD-REPO01 and register every path it uses.

Follow the build

1. Write the repository's job: the only source of operating-system packages for the domain-joined Linux servers (SBD-SPLUNK01 in course 1.12, SBD-WEB01 in course 1.18, and SBD-NESSUS01 when course 1.17, coming soon, arrives) and for SBD-REPO01 itself. Application software, such as Splunk in course 1.12 and SCC for Linux in lesson 5, still arrives as hashed vendor media; anything else is a documented exception, never a second source.
2. Choose one Red Hat Enterprise Linux–family distribution and major release for every Linux server, one with a published DISA STIG and a SCAP benchmark that SCC for Linux supports, so there is one mirror to feed and one benchmark to scan against. Mirror only its operating-system repositories for one architecture: BaseOS and AppStream. Third-party repositories stay out.
3. Decide the trust anchor: the vendor's signing keys. Get each key file, read its fingerprint, and compare it with the vendor's published value through a second channel before anything is published; the fingerprints go into the snapshot catalog. Each published mirror is a dated snapshot, and that date is the repository's version.
4. Place it as the System Hub does: SBD-REPO01 at 10.0.70.20 on VLAN 70, not domain-joined, administered over SSH 22 from SBD-ADMIN-SRV01 only (rule A02, Tier 1). Confirm the register rows: the Linux servers to REPO01 on HTTP 80 (F19), REPO01 to both domain controllers for DNS (F01), and REPO01's syslog over TLS to the collector (F07, enabled when course 1.10 builds it). Rule F09 lists only domain members for time, so add SBD-REPO01's own time row, to both controllers on UDP 123, with an owner and a reason, as course 1.3 planned.

Check your result

```
# On SBD-ADMIN-SRV01: start the evidence folder and confirm no record exists yet
New-Item -ItemType Directory -Force C:\Evidence\linux-repository | Out-Null
Resolve-DnsName SBD-REPO01.sbd.test -Server 10.0.10.10 -ErrorAction SilentlyContinue   # expected: no answer yet
# On the approved connected system: read each upstream signing key's fingerprint
gpg --show-keys --with-fingerprint /etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release   # or the distribution's own key file
```

WHAT SUCCESS LOOKS LIKE The design names the consumers, the one distribution and the repositories mirrored; each upstream key's fingerprint matches the vendor's published value; and the register carries F19, F01, F07 and SBD-REPO01's added time row, each with an owner.

Your record

Evidence to keep: Keep the repository design (job, consumers, distribution, repositories, snapshot naming), the verified key fingerprints with where you checked them, and the register rows that name SBD-REPO01.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why does this course come before any Linux server is built?

REMEMBER One distribution, one mirror, one source of Linux packages. Verified upstream keys are the trust anchor; the dated snapshot is the version. REPO01's place and every path it uses are in the register, including its time row.

Red Hat · Managing software with the DNF tool

Red Hat · Product signing keys

NIST · SP 800-53 Rev. 5 security controls

LESSON 02 / ABOUT 40 MIN

Build and harden SBD-REPO01

> **WHY THIS MATTERS** Build SBD-REPO01 as a minimal Linux server that does one job: answer package requests on TCP 80 and accept administration on SSH from one seat. Every other service, listener and account is left out.

In this lesson you will

- Install the server with its static address and DNS record, course 1.3's chrony file, and the course 1.6 root in its trust store.
- Restrict administration to named local accounts over SSH from SBD-ADMIN-SRV01.
- Serve only the repository directory from a minimal web server behind a host firewall that mirrors the register.

Follow the build

1. Install the chosen distribution's minimal server profile from install media whose hash matches the vendor's published checksum, with a separate file system mounted at /srv, sized for at least two snapshots plus growth: /srv/repo is the web root, /srv/incoming receives each crossing, and /srv/repo-records holds the catalog. Set the name SBD-REPO01.sbd.test and the static address 10.0.70.20/24, with the core's VLAN 70 interface (10.0.70.1 in this sample) as gateway and both domain controllers as DNS servers. Because REPO01 cannot register itself in DNS, the DNS owner adds its A record (and PTR, if the reverse zone exists) from AdminJumpBox01. Install only from that verified media until SBD-REPO01 serves its own snapshot; if the image lacks a package this lesson needs, such as chrony or the web server, bring it signature-checked with the first crossing (lesson 3) and install it from that copy.
2. Apply course 1.3's chrony file before the first package update, and turn off the hypervisor's time synchronization for the VM, so the domain controllers are its only clock: /etc/chrony.conf names only the two controllers, server SBD-DC01.sbd.test iburst and server SBD-DC02.sbd.test iburst, with maxdistance 16 and no pool, other server, sourcedir or allow line, and reaches both controllers over the time row added in lesson 1. Check that chronyc sources shows one controller selected (^*) and the other usable, with a small, steady offset; the maxdistance line is what admits controllers that serve their own clock, as course 1.3 explains.
3. Create a named local account for each administrator in one admin group with sudo, lock root's password, and copy each administrator's SSH public key from SBD-ADMIN-SRV01. In a drop-in that sorts before the distribution's own (for example /etc/ssh/sshd_config.d/10-sbd.conf, because the first value read wins), set PermitRootLogin no, PasswordAuthentication no and AllowGroups to that group. Over your first SSH session, add the course 1.6 root certificate to the system trust store, because a server outside the domain gets it from no Group Policy: export it from SBD-ADMIN-SRV01's Trusted Root store, check its SHA-256 against the hash course 1.6 recorded, copy it across, place it in /etc/pki/ca-trust/source/anchors and run update-ca-trust, ready for course 1.10's syslog over TLS.
4. Install Apache (httpd) and give it one site: Listen 10.0.70.20:80, DocumentRoot /srv/repo, Options FollowSymLinks with no Indexes, AllowOverride None, only GET and HEAD allowed, everything outside /srv/repo denied, ServerTokens Prod, ServerSignature Off and TraceEnable Off. Remove the welcome page, autoindex and user directories; with SELinux enforcing, label /srv/repo httpd_sys_content_t with semanage fcontext so restorecon keeps it. The repository files stay owned by root and read-only to the web server.

© 2026 Secure By Design Team LLC.

5. Make the host firewall the register's copy. With firewalld, create a zone for 10.0.60.20 that allows only ssh and a zone for SBD-SPLUNK01, SBD-NESSUS01 and SBD-WEB01 (10.0.50.10, 10.0.50.20, 10.0.40.40) that allows only http, with no services left in the default zone. Adding those consumers now, before they exist, is safe because F19 already approves them.

Check your result

```
hostnamectl; ip -4 addr show
grep -E '^(pool|server|sourcedir|allow|maxdistance)' /etc/chrony.conf   # both controllers
and maxdistance 16, nothing else
chronyc sources -v; chronyc tracking   # ^* one controller, ^+ or ^- the other; System
time offset small and steady
sshd -T | grep -Ei '^(permitrootlogin|passwordauthentication|allowgroups) '
# On SBD-ADMIN-SRV01: export the course 1.6 root and compare its hash with course 1.6's
record, then copy it over
Get-ChildItem Cert:\LocalMachine\Root | Where-Object Subject -like '*<root CA name>*' |
Export-Certificate -FilePath C:\Evidence\linux-repository\sbd-root.cer
Get-FileHash -Algorithm SHA256 C:\Evidence\linux-repository\sbd-root.cer
scp C:\Evidence\linux-repository\sbd-root.cer <your-account>@SBD-REPO01.sbd.test:/tmp/
# On SBD-REPO01: the same fingerprint, then the system trust store
openssl x509 -inform der -in /tmp/sbd-root.cer -noout -fingerprint -sha256
sudo cp /tmp/sbd-root.cer /etc/pki/ca-trust/source/anchors/ && sudo update-ca-trust
trust list --filter=ca-anchors | grep -i '<root CA name>'   # expected: the SBD root
apachectl -S        # expected: one site on 10.0.70.20:80
ss -tlnp            # expected: 22 and 80, plus loopback-only listeners, if any
firewall-cmd --get-active-zones
firewall-cmd --list-all --zone=public   # expected: no services
curl -sI http://SBD-REPO01.sbd.test/ | head -1   # expected: 403, no directory listing
```

> **WHAT SUCCESS LOOKS LIKE** SBD-REPO01 resolves to 10.0.70.20 and, with course 1.3's chrony file and the hypervisor's time synchronization off, syncs from a domain controller with a small, stable offset; sshd refuses root and passwords and admits only the admin group; the system trust store holds the course 1.6 root, its hash matching course 1.6's record; only TCP 22 and 80 listen; the firewall admits SSH from 10.0.60.20 alone and HTTP from the three registered consumers alone; and the web root returns 403 instead of a listing.

Your record

Evidence to keep: Keep the install media hash, the chrony file with its sources and tracking output, the root certificate's hash compared with course 1.6's record, the effective sshd settings, the Apache site and listener output, the firewall zones, and the administrator account list (names only, no keys or passwords).

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	

RECORD	YOUR OBSERVATION
Open items or findings	

Think it through

Why does chrony on SBD-REPO01 point at both domain controllers?

> **REMEMBER** A minimal install: static address, DNS record, and time from both controllers through course 1.3's chrony file. Named accounts, keys only, SSH from one seat, and the SBD root trusted. One web site, one directory, and a firewall that matches the register.

Red Hat · Using shared system certificates

Apache HTTP Server 2.4 · Security tips

OpenSSH · sshd_config(5)

chrony · chrony.conf(5)

firewalld · documentation

Microsoft · Add-DnsServerResourceRecordA

Microsoft · Export-Certificate

NIST · SP 800-53 Rev. 5 security controls

LESSON 03 / ABOUT 35 MIN

Mirror, verify, and publish a snapshot

> **WHY THIS MATTERS** The repository's content crosses the boundary like everything else: mirrored on the connected side, hashed, carried across, and checked twice on arrival, once against the hash manifest and once against the vendor's signatures. Only then is it published as a dated snapshot and entered in the catalog.

In this lesson you will

- Mirror the approved repositories with their signatures checked on the connected side.
- Carry the set across and verify both its hashes and its signatures before publishing.
- Publish a dated snapshot and record it in the snapshot catalog.

Follow the build

1. On the approved connected system, mirror into a dated directory. From a registered system of the same release, run dnf reposync with --download-metadata, which keeps the vendor's own repository metadata (including groups and modules) so the copy is usable as a repository as it stands, and --gpgcheck, which deletes any package whose signature fails and exits non-zero.
2. If you trim the set instead of mirroring it whole, the vendor metadata no longer matches the files: rebuild it with createrepo_c (and add module metadata back with modifyrepo_c) and rely on gpgcheck on every client, because your rebuilt metadata carries no vendor signature. Then write a SHA-256 manifest of every file, hash the manifest itself, and carry both across under the course 1.4 media procedure.
3. On SBD-REPO01, copy the set and its manifest into /srv/incoming, outside the web root, and check the set against the manifest. Then verify signatures independently of the connected side: rpm -K on every package, against the keys verified in lesson 1 (rpm --import those key files first if rpm -q gpg-pubkey does not list them). One failure stops publication and becomes a finding.
4. Publish: move the set to /srv/repo/snapshots/<date> (a rename on the same file system), restore its SELinux labels, and switch the current link to it with an atomic rename so clients never see a half-copied mix. The previous snapshot stays in place for rollback.
5. Add the snapshot to the catalog in /srv/repo-records, outside the web root: date, repositories, upstream key fingerprints, manifest hash, transfer record, who published it, and its status (current, previous or retired). Copy the catalog and manifest to C:\Evidence\linux-repository on SBD-ADMIN-SRV01 and hash them.

Check your result

```
# Connected side: keep vendor metadata, delete anything whose signature fails
dnf reposync --repoid=<baseos-repo-id> --repoid=<appstream-repo-id> --download-metadata --
gpgcheck -p /srv/stage/2026-10-13
cd /srv/stage/2026-10-13 && find . -type f -print0 | sort -z | xargs -0 sha256sum >
../SHA256SUMS-2026-10-13.txt
# On SBD-REPO01, after the crossing:
cd /srv/incoming/2026-10-13 && sha256sum -c --quiet ../SHA256SUMS-2026-10-13.txt   #
expected: no output
find . -name '*.rpm' -print0 | xargs -0 rpm -K | grep -v 'digests signatures OK'   #
expected: no output
mv /srv/incoming/2026-10-13 /srv/repo/snapshots/ && restorecon -R
/srv/repo/snapshots/2026-10-13
```

```
ln -sfn snapshots/2026-10-13 /srv/repo/current.new && mv -T /srv/repo/current.new
/srv/repo/current
```

> **WHAT SUCCESS LOOKS LIKE** The manifest check and the signature check both pass with no exceptions, /srv/repo/current points at the new dated snapshot, the previous snapshot (if any) is untouched, and the catalog entry carries the fingerprints and manifest hash.

Your record

Evidence to keep: Keep the manifest and its hash, the transfer record, the clean signature-check output, the catalog entry, and the hashes of the copies filed on SBD-ADMIN-SRV01.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What is the repository's version, for evidence purposes?

> **REMEMBER** Mirror with signature checks on, then hash the whole set. On arrival, check the hashes and the signatures; either failure stops publication. A dated snapshot goes live by atomic switch and enters the catalog.

DNF plugins · reposync

createrepo_c · repository metadata tools

Red Hat · Product signing keys

Microsoft · Get-FileHash

LESSON 04 / ABOUT 30 MIN

Configure the Linux clients

> **WHY THIS MATTERS** Write the one client configuration every Linux server will use: a single source definition pointing at SBD-REPO01, the verified signing keys, and no other source. No consumer exists yet, so prove it on SBD-REPO01's own package manager and hand the files to courses 1.12 and 1.18.

In this lesson you will

- Write the dnf source definition and signature settings for the chosen release.
- Trust only the verified keys and remove every other package source.
- Prove the configuration on SBD-REPO01 and package it for the later Linux servers.

Follow the build

1. Write one source definition that names SBD-REPO01.sbd.test, so DNS carries it (F01): /etc/yum.repos.d/sbd-repo01.repo has one section per repository with baseurl=http://SBD-REPO01.sbd.test/current/<repo-id>/, gpgcheck=1 and gpgkey=file:///etc/pki/rpm-gpg/<vendor key>.
2. Trust only the keys verified in lesson 1, installed from local files and never fetched from the repository at install time: whoever could change the repository could then change the key too. List the trusted keys (rpm -q gpg-pubkey) and remove any key nothing needs with rpm -e.
3. Remove or disable every other source: set enabled=0 in every other .repo file, including any install-media entry, and where subscription-manager is installed, stop it managing repositories (subscription-manager config --rhsm.manage_repos=0), so it cannot write its own definitions back.
4. Prove it on SBD-REPO01 itself, which follows the same rule: apply the files, refresh the metadata with no errors or warnings, and confirm that a package you will need in course 1.10, rsyslog-gnutls, resolves from the SBD-REPO01 source. A Failed to fetch line or a missing package is a gap in the mirror to fix before any server depends on it.
5. Package the source file, the key files and a short apply-and-verify sheet in /srv/repo-records/client, hash them, and file them as the configuration every Linux server applies, courses 1.12 and 1.18 when they build SBD-SPLUNK01 and SBD-WEB01: after course 1.3's chrony file and before the server's first package update.

Check your result

```
dnf repolist --all                 # expected: only the sbd-repo01 repositories enabled
rpm -q gpg-pubkey --qf '%{NAME}-%{VERSION}-%{RELEASE}\t%{SUMMARY}\n'
dnf makecache && dnf info rsyslog-gnutls | grep -Ei '^(repository|from repo)'   #
expected: from the sbd-repo01 repositories
# Hash the client package for the later courses
sha256sum /srv/repo-records/client/*
```

> **WHAT SUCCESS LOOKS LIKE** On SBD-REPO01 the package manager lists SBD-REPO01 as its only source, trusts only the verified keys, refreshes without errors, and resolves rsyslog-gnutls from the snapshot; the client package is hashed and filed.

Your record

Evidence to keep: Keep the source listing, the trusted key list, the refresh output, the package's source line, and the client package with its hashes.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

How is the one-source rule enforced on a Linux consumer?

REMEMBER One source definition, by name, with signature checks on. Only verified keys are trusted, and no other source remains. Proven on SBD-REPO01, packaged for courses 1.12 and 1.18.

Red Hat · Managing software with the DNF tool

Red Hat · Product signing keys

NIST · SP 800-53 Rev. 5 security controls

LESSON 05 / ABOUT 40 MIN

Keep it current and prove it

> **WHY THIS MATTERS** A repository is only as good as its last snapshot. Set the cadence that keeps it current, rehearse promoting a new snapshot and rolling back, stage SCC for Linux for every Linux server and take SBD-REPO01's first SCAP baseline with it, and close the course with a record an assessor can check.

In this lesson you will

- Set the update cadence for snapshots across the air gap.
- Promote a new snapshot and roll back to the previous one, on record.
- Stage SCC for Linux and the STIG benchmark for every Linux server, baseline SBD-REPO01 with them, and hand off to course 1.10.

Follow the build

1. Write the cadence: a new snapshot crosses on a fixed schedule (monthly is a common sample value), plus an expedited crossing when the vendor publishes a critical security fix. Keep the current and previous snapshots, retire older ones in the catalog, and name who approves each crossing. Course 1.14 later carries Windows updates on the same monthly run.
2. Rehearse change control with the next snapshot: publish it as in lesson 3, refresh SBD-REPO01's own client and confirm it sees the new content, then switch current back to the previous snapshot and confirm the client follows. Rolling back stops a bad snapshot from spreading, but it does not uninstall anything; a server that already updated needs dnf history undo or dnf downgrade there. Record both switches in the catalog.
3. Stage the Linux assessment set once, for every Linux server: SCC for Linux for your distribution and architecture, the DISA STIG SCAP benchmark for the distribution, and, for the manual items, the distribution's STIG and the Apache Server 2.4 UNIX STIGs. Check SCC against DISA's published checksum on the connected side, hash every file into connected-side-hashes.csv, carry the set under the course 1.4 media procedure, and check it again into C:\Staging\DISA\linux on SBD-ADMIN-SRV01: the Tier 1 copy that courses 1.12 and 1.18 scan their Linux servers with.
4. Scan every Linux server the same way, starting with SBD-REPO01: copy SCC and the benchmark over your Tier 1 SSH session, check their hashes there, install SCC as its readme directs, and run it as root with the benchmark and the profile course 1.4 recorded. Copy the session back to C:\Evidence\linux-repository\baseline\SBD-REPO01, answer the manual items, Apache's included, in a STIG Viewer checklist for SBD-REPO01 on SBD-ADMIN-SRV01, and classify every open item as in course 1.4: fix now, deviation candidate or POA&M candidate. Plain HTTP on F19 is a deviation candidate, justified by signed content.
5. File the assessable record in C:\Evidence\linux-repository with hashes: the design, the snapshot catalog, the client package, the baseline results and checklist, and the administration record of who changed what. Write the handoff to course 1.10: SBD-REPO01 is running, rsyslog-gnutls is in its snapshot, the SBD root is in its trust store, and its syslog over TLS (F07) is waiting for the collector.

Check your result

```
# On SBD-REPO01: promote the next snapshot, then rehearse rollback
readlink /srv/repo/current
```

```
ln -sfn snapshots/2026-11-10 /srv/repo/current.new && mv -T /srv/repo/current.new
/srv/repo/current
ln -sfn snapshots/2026-10-13 /srv/repo/current.new && mv -T /srv/repo/current.new
/srv/repo/current   # rollback
# On SBD-ADMIN-SRV01: no output means every staged file is on the connected-side list
(Get-ChildItem C:\Staging\DISA\linux\* -File -Exclude *.csv | Get-FileHash -Algorithm
SHA256).Hash | Where-Object { $_ -notin (Import-Csv C:\Staging\DISA\linux\connected-side-
hashes.csv).Hash }
scp C:\Staging\DISA\linux\<SCC for Linux package> C:\Staging\DISA\linux\<STIG benchmark>
<your-account>@SBD-REPO01.sbd.test:/tmp/
# On SBD-REPO01: the same hashes, then SCC's command-line scanner as root (path and
options from the SCC readme)
sha256sum /tmp/<SCC for Linux package> /tmp/<STIG benchmark>
sudo <SCC install directory>/cscc
# On SBD-ADMIN-SRV01: collect the session and the record, then hash everything
New-Item -ItemType Directory -Force C:\Evidence\linux-repository\baseline | Out-Null
scp -r <your-account>@SBD-REPO01.sbd.test:<SCC session folder> C:\Evidence\linux-
repository\baseline\SBD-REPO01
scp -r <your-account>@SBD-REPO01.sbd.test:/srv/repo-records/* C:\Evidence\linux-
repository\
Get-ChildItem C:\Evidence\linux-repository -Recurse -File | Get-FileHash -Algorithm SHA256
| Export-Csv C:\Evidence\linux-repository-hashes.csv -NoTypeInformation
```

WHAT SUCCESS LOOKS LIKE The cadence is written with an approver, the promotion and the rollback both show in the catalog and in the client's view, the Linux assessment set sits checked in C:\Staging\DISA\linux on SBD-ADMIN-SRV01, SBD-REPO01 has a first baseline with every open item classified, and the record and handoff are filed with hashes.

Your record

Evidence to keep: Keep the cadence, the catalog showing promotion and rollback, the Linux assessment set's hash check, the SCC results and STIG Viewer checklist, the classified findings, the administration record, the hash list, and the handoff note to course 1.10.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

You roll the repository back to the previous snapshot. What happens to servers that already installed the newer packages?

REMEMBER Snapshots cross on a written cadence, with an expedited path for critical fixes. Promotion and rollback are rehearsed, and rollback's limits are known. SCC for Linux is staged for every Linux server, SBD-REPO01 has its first baseline, and course 1.10 inherits a running source.

DISA · Security Content Automation Protocol (SCC and SCAP benchmarks)

DISA · STIGs document library

DISA · SRG/STIG tools (SCC and STIG Viewer)

Microsoft · Get-FileHash

NIST · SP 800-53 Rev. 5 security controls

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Design the repository and its trust	☐ yes	
02 Build and harden SBD-REPO01	☐ yes	
03 Mirror, verify, and publish a snapshot	☐ yes	
04 Configure the Linux clients	☐ yes	
05 Keep it current and prove it	☐ yes	

Open items carried forward

Next in the learning path

That completes course 1.9. Course 1.10 builds the collector, SBD-SYSLOG01, and enrolls SBD-REPO01 as its first Linux syslog-over-TLS source.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.